10 November 2026

Struktur Data

Tree: Konsep & Implementasi



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

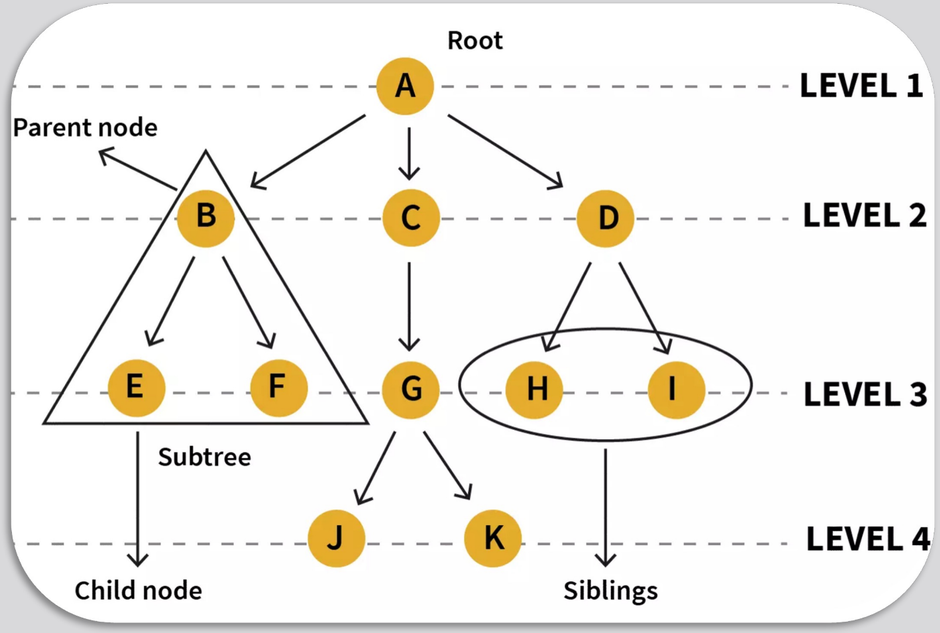
Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta

Definisi Tree

Tree adalah **struktur data non-linear** yang **merepresentasikan hubungan hierarkis antar elemen** melalui node dengan hubungan parent-child. **Node utama** disebut **root**, **node tanpa anak** disebut **leaf**, dan **setiap node** (kecuali root) memiliki **satu parent**.

Berbeda dari array atau linked list, **tree cocok** untuk **data hirarkis**, seperti sistem file, ekspresi dalam pemrograman, dan **algoritma pencarian** atau **pengurutan**, memudahkan navigasi dan **manipulasi data kompleks**.



Karakteristik Tree

Terdapat beberapa **karakteristik struktur data tree**:

- ✓ **Terdiri** dari **node** yang **menyimpan data**.
- ✓ **Memiliki root (akar)** sebagai **node utama**.
- ✓ **Setiap node (kecuali root)** memiliki **parent (induk)**.
- ✓ **Node** dapat memiliki **child (anak)**.
- ✓ **Node** yang **tidak memiliki child** disebut **leaf (daun)**.
- ✓ **Tidak ada siklus (cycle)** dalam Tree.

Istilah Penting Tree

Terdapat beberapa **istilah** yang **penting** terkait dengan **tree**:

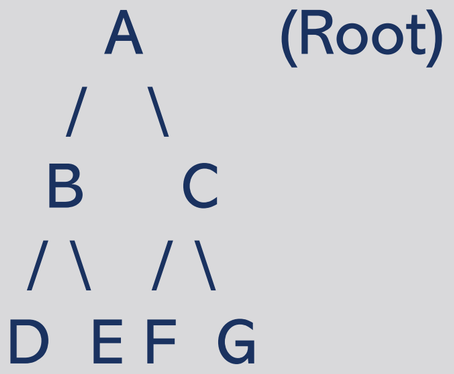
- ✓ **Node**: Unit dasar dalam tree yang menyimpan data.
- ✓ **Root**: node paling atas dalam tree; setiap tree hanya memiliki satu root.
- ✓ **Child**: Node yang terhubung langsung ke node lain di bawahnya.
- ✓ **Parent**: Node yang menjadi induk bagi node-node di bawahnya.
- ✓ **Leaf Node**: Node yang tidak memiliki anak (node terminal)

Istilah Penting Tree

- ✓ **Height dari Node**: Jarak terpanjang dari node tersebut ke leaf node.
- ✓ **Depth dari Node**: Jumlah edge antara node tersebut dan root.
- ✓ **Subtree**: Struktur tree yang terdiri dari node tertentu dan semua keturunannya

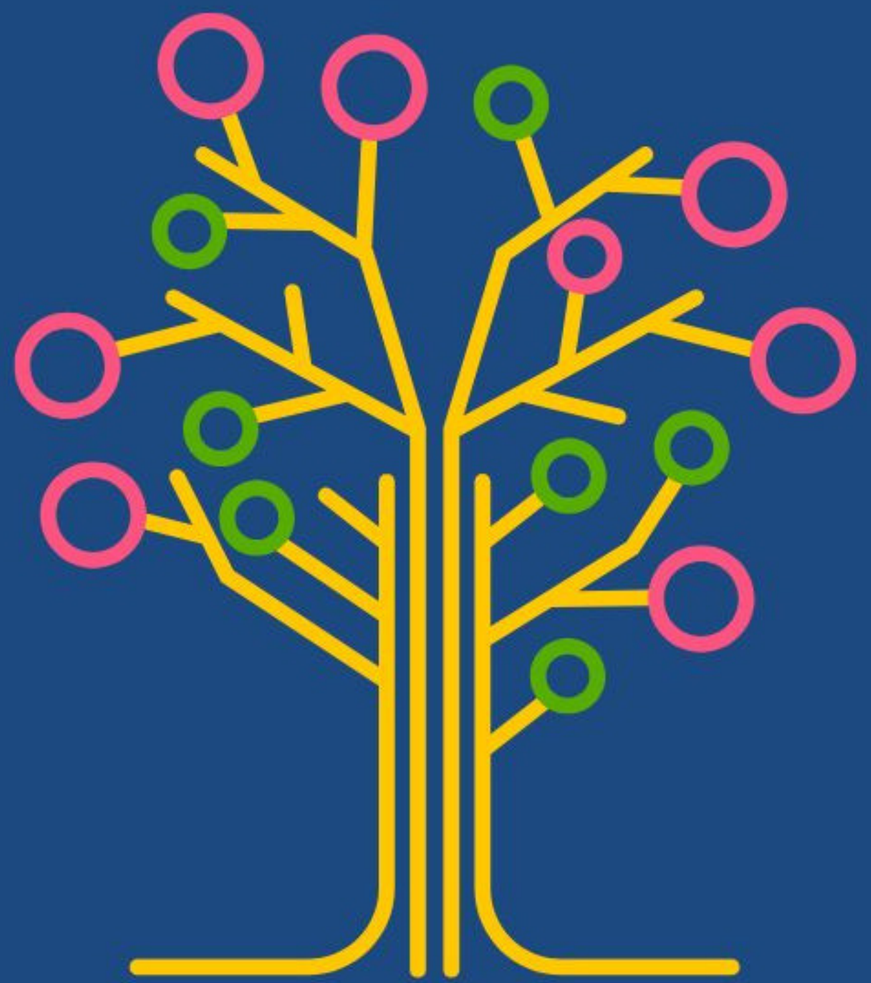
Struktur Dasar Tree

Berikut adalah **representasi sederhana** dari sebuah **tree**:



Dalam contoh di atas:

- ✓ **Node A** adalah **root**.
- ✓ **Node B** dan **C** adalah **children** dari **A**, sedangkan **D, E, F, dan G** adalah **leaf nodes**.



Jenis-Jenis
Tree

1. Binary Tree
2. Binary Search Tree (BST)
3. AVL Tree
4. Red-Black Tree
5. N-ary Tree

Jenis-Jenis Tree

1. **Binary Tree**: Tree di mana setiap node memiliki maksimal dua anak (left dan right). Dalam binary tree, satu dari anak-anak tersebut bisa saja tidak ada.

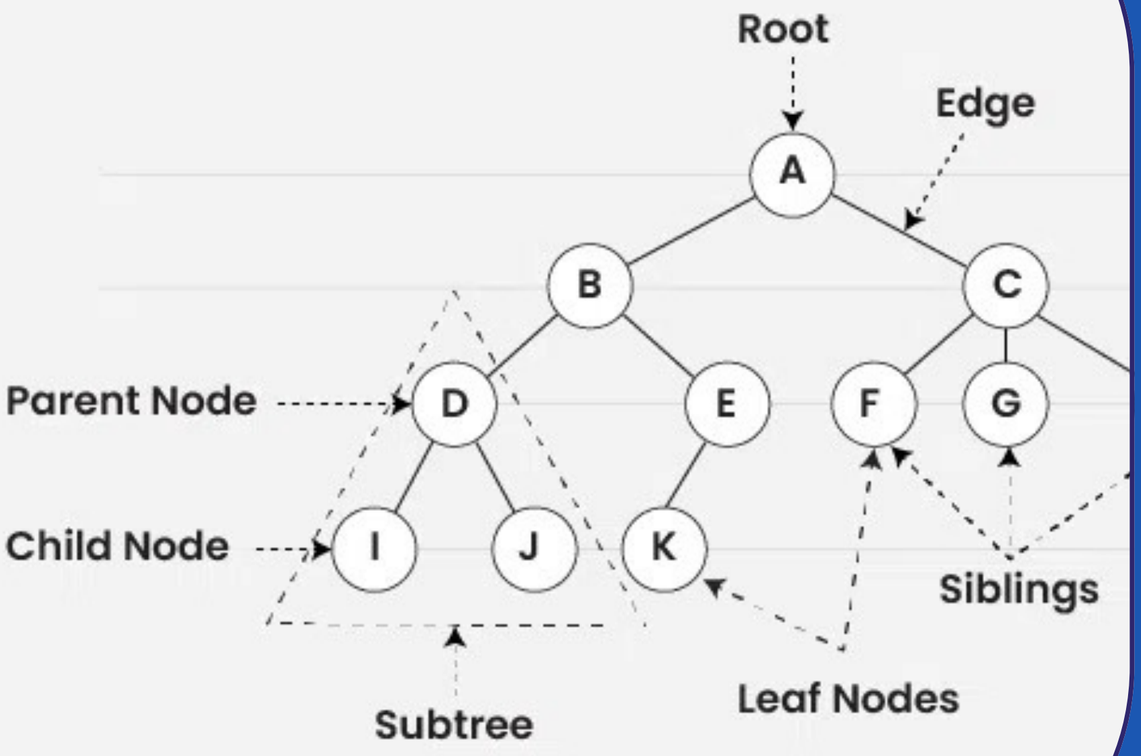
2. **Binary Search Tree**: Tipe binary tree di mana nilai node di subtree kiri selalu lebih kecil dari nilai node, dan nilai node di subtree kanan selalu lebih besar. Ini memudahkan pencarian elemen.

3. **AVL Tree**: Jenis dari binary search tree yang juga menjaga keseimbangan. Setiap node di tree memiliki balance factor yang harus berada dalam rentang -1, 0, atau 1. Ini menjaga performa pencarian, penyisipan, dan penghapusan agar tetap O(log n).

Jenis-Jenis Tree

4. **Red-Black Tree**: Jenis binary search tree lain yang juga menjaga keseimbangan, tetapi dengan aturan warna untuk node-nya (merah dan hitam). Ini memastikan bahwa pohon tetap seimbang di berbagai operasi.

5. **N-ary Tree**: Pohon di mana setiap node dapat memiliki N anak. Ini lebih umum dan tidak dibatasi pada dua anak.



Operasi Dasar Tree

1. Traversal
2. Penyisipan (Insertion)
3. Penghapusan (Deletion)

Operasi Traversal

Traversal adalah cara untuk **mengunjungi setiap node** dalam tree. Ada beberapa **metode** untuk **traversing** sebuah tree:

- ✓ **Pre-order Traversal**: Kunjungi node, kemudian subtree kiri, lalu subtree kanan.

- ✓ **In-order Traversal**: Kunjungi subtree kiri, node, lalu subtree kanan. (Berguna untuk Binary Search Tree untuk mendapatkan elemen terurut)

- ✓ **Post-order Traversal**: Kunjungi subtree kiri, subtree kanan, lalu node.

- ✓ **Level-order Traversal**: Kunjungi setiap level tree dari atas ke bawah.

Operasi Penyisipan

Penyisipan node baru ke dalam **tree** tergantung pada **jenis tree**. Berikut deskripsi **langkah-langkah operasi penyisipan** pada struktur data tree (misalnya **binary tree**):

- ✓ Mulai dari **root**.
- ✓ **Bandingkan nilai** dengan node saat ini.
- ✓ Lanjut ke **subtree kiri** jika **lebih kecil**, **subtree kanan** jika **lebih besar**.
- ✓ **Cari posisi kosong**.
- ✓ **Sisipkan node baru** sebagai **child**, **perbarui** hubungan **parent-child**.

Operasi Penghapusan

Berikut deskripsi **langkah-langkah operasi penghapusan** pada **tree** (misalnya **binary search tree – BST**):

- ✓ Mulai dari **root** dan cari **node** yang akan **dihapus**.
- ✓ **Tentukan kasus**: leaf, satu child, atau dua child.
- ✓ Jika **leaf**, **hapus langsung**.
- ✓ Jika satu **child**, **hubungkan child** dengan **parent node**.
- ✓ Jika **dua child**, **ganti** dengan **predecessor** atau **successor**, lalu **hapus node pengganti**, pastikan struktur tree tetap valid.



Aplikasi
Tree

1. Representasi Hierarkis
2. Database & Sistem Pengindeksan
3. Pengolahan Ekspresi dalam Komputasi
4. Artificial Intelligence dan Algoritma Pencarian
5. Kompresi Data

Aplikasi Tree

1. **Representasi Hierarkis**: Tree efektif merepresentasikan data bertingkat, seperti sistem file, organisasi perusahaan, silsilah keluarga, atau dokumen XML/HTML, memudahkan navigasi, pengelompokan, dan pengelolaan data.

2. **Database dan Pengindeksan**: Tree, seperti B-Tree dan B+ Tree, digunakan dalam DBMS untuk pengindeksan, mempercepat pencarian, penyisipan, dan penghapusan data, serta mengoptimalkan akses disk pada database besar.

3. **Pengolahan Ekspresi**: Binary tree digunakan untuk parsing dan evaluasi ekspresi, dengan node internal sebagai operator dan leaf sebagai operand, memudahkan konversi infix-postfix serta optimisasi kompilasi.

Aplikasi Tree

4. **Artificial Intelligence dan Pencarian**: Tree digunakan untuk memodelkan state space dalam algoritma pencarian, seperti search tree dan decision tree, membantu pemecahan masalah, game AI, serta klasifikasi dan prediksi dalam machine learning.

5. **Kompresi Data**: Tree, seperti Huffman Tree, digunakan dalam Huffman Coding untuk memberikan kode biner variabel berdasarkan frekuensi simbol, sehingga data dapat dikompresi efisien pada file JPEG, MP3, dan ZIP.

Kelebihan Tree

1. **Pengorganisasian Data yang Baik**: Tree membantu dalam pengorganisasian data secara hierarkis dan efisien.

2. **Akses Cepat**: Operasi pencarian, penyisipan, dan penghapusan dapat dilakukan dengan waktu yang efisien (O(log n) untuk BST yang seimbang).

3. **Memudahkan Traversal Data**: Memberikan berbagai cara untuk mengunjungi dan memanipulasi data.

Kekurangan Tree

1. **Memori**: Memerlukan lebih banyak memori dibandingkan dengan struktur data linier seperti array.

2. **Kemuatan Seimbang**: Kinerja dari tree tergantung pada sejauh mana tree seimbang. Tree yang tidak seimbang dapat menyebabkan penurunan performa hingga O(n).

10 November 2026

Sekian

Terima Kasih



Andi Nurkholis, S.Kom., M.Kom.

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta